

EXPLAINABILITY REPORT

AI-Powered Internship Performance Prediction & Risk Analytics Platform

Ezitech Engineering Framework (EEF)
Industry AI Case Study AI-005

Deliverable: Explainability Report
Explainability Method: SHAP (SHapley Additive exPlanations)

Prepared by
Sadaf Riaz & Saira Ejaz

This report documents how model predictions on this platform are made interpretable — at both a global (model-wide) and local (per-intern) level — how those explanations are generated, stored, and surfaced to Admins, Mentors, and Students, and where the current implementation's limitations lie.

1. Purpose & Approach

Predictions that mentors and admins cannot interrogate do not get trusted or acted on. If the platform reports that an intern has a 78% dropout risk, the mentor needs to know *why* before that number becomes a useful conversation with the intern. This report documents how explainability is implemented across the platform's four trained models, how per-prediction explanations are generated and served through the REST API, and how they are exposed in the dashboards.

Technique used: SHAP (SHapley Additive exPlanations), specifically `shap.TreeExplainer`, applied to the three tree-based XGBoost models in the system. SHAP was chosen because a single, theoretically grounded method produces both a global view (which features matter most across the whole intern population) and a local view (why this one intern received this one score), and it integrates natively with XGBoost with no extra surrogate modeling required.

1.1 Two Levels of Explainability

Level	Where it lives	Question it answers
Global	<code>train_dropout_model.py</code> , <code>train_success_model.py</code> — SHAP summary plots generated once, right after training, on the held-out test set.	Across all interns, which features drive this model's decisions the most, and in which direction?
Local (per-prediction)	<code>app/ml/predict.py</code> → <code>_shap_explanation()</code> — computed live, on every API call, for one intern's feature row.	For this specific intern, what were the top 3 factors pushing their score up or down?

Every prediction returned by a `/predict/*` endpoint includes an explanation object built by `_shap_explanation()`. It ranks each feature's SHAP value for that one intern by absolute magnitude and returns the top 3, each with its signed contribution:

```
"top_factors": [  
  {"feature": "avg_code_review_score", "contribution": -1.82},  
  {"feature": "attendance_trend", "contribution": 0.64},  
  {"feature": "avg_mentor_rating", "contribution": -0.31}  
]
```

A negative contribution pushes the prediction toward "lower risk / less likely"; a positive one pushes it toward "higher risk / more likely." The sign follows SHAP's own output for the class that was actually predicted — not a custom scale.

The function also handles the two SHAP output shapes seen in practice: for the binary models (Dropout Risk, Success Probability) it takes the single class-1 array; for the 3-class Performance Trend model it explicitly slices out the array for whichever class was actually predicted — SHAP returns one array per class for multi-class models, and blindly taking the first would silently explain the wrong class. This was a real issue encountered during development and is now guarded with a `class_index` parameter plus a fallback for both the list-of-arrays and 3D-array SHAP return formats.

2. Model 1 — Dropout Risk

XGBoost binary classifier • Global + local SHAP explainability implemented

Global explainability: generated as shap_summary.png at the end of train_dropout_model.py, on the held-out test set.

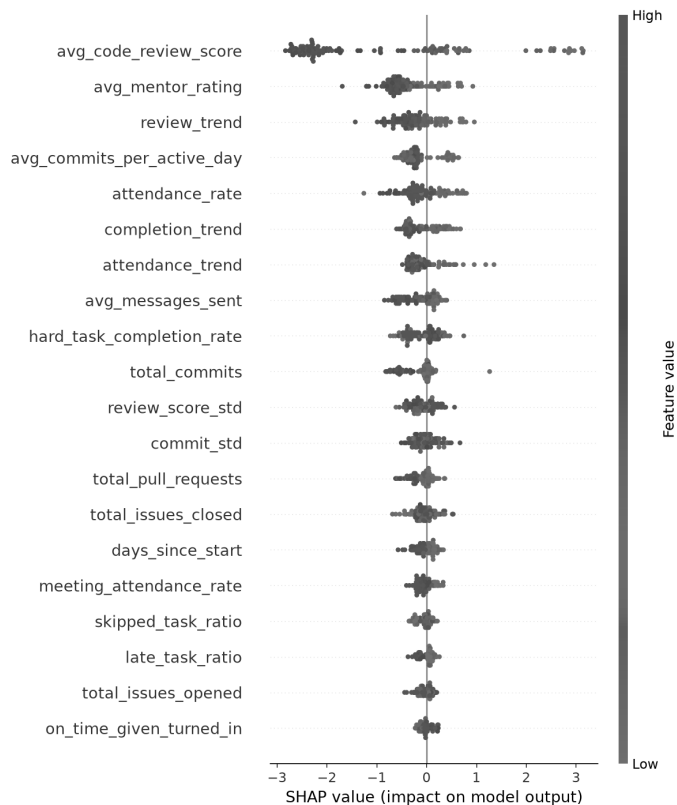


Figure 1 — SHAP summary plot, Dropout Risk model (grayscale). Each point is one intern in the test set; horizontal position is that feature's impact on the predicted risk.

Top drivers, ranked by mean |SHAP value|:

Rank	Feature	Direction of effect
1	avg_code_review_score	Dominant. Low scores push risk sharply up; high scores push it down.
2	avg_mentor_rating	Low mentor ratings increase predicted dropout risk.
3	review_trend	A declining review-score trend raises risk, independent of the average level.
4-7	avg_commits_per_active_day, attendance_rate, completion_trend, attendance_trend	Moderate, consistent effects — low/declining values push risk up.

Rank	Feature	Direction of effect
8+	avg_messages_sent, hard_task_completion_rate, total_commits, review_score_std, commit_std, total_pull_requests, total_issues_closed, days_since_start, meeting_attendance_rate, skipped_task_ratio, late_task_ratio, total_issues_opened, on_time_given_turned_in	Small individual effect sizes.

Business read: the model leans most heavily on *quality and feedback signals* (review scores, mentor ratings) and *trajectory* (getting better or worse), not just raw activity counts. This matches the design intent of the trend features in features.py: dropout probability is tied to whether an intern's trajectory is declining, so a consistently-mediocre-but-stable intern is flagged as less risky than one who is clearly sliding.

Local explainability: every call to predict_dropout_risk() also returns the intern's own top-3 SHAP factors, plus the model's tuned decision threshold (0.45, selected by F1 during training rather than the default 0.5) and a boolean flagged_as_at_risk.

3. Model 2 — Performance Trend

XGBoost 3-class classifier (declining / stable / improving) • Local SHAP implemented; global SHAP artifact mismatched

Local explainability: fully implemented and live. Every prediction returns top_factors for the *predicted* class specifically (via the class_index handling described in Section 1.1), plus the full 3-class probability distribution (class_probabilities) — so a mentor can see how close a "stable" call was to "declining."

3.1 Known Gap: Global Explainability Artifact Mismatch

The training script (train_performance_model.py) does **not** generate a SHAP summary plot at all — there is no shap.TreeExplainer call or savefig() in that file. The image currently saved as shap_summary_performance.png does not actually belong to this model: its feature names (completion_early, review_early, commits_early, attendance_early) match the **Learning Speed / Skill Growth** model's early-period feature set (build_growth_dataset.py), not Performance Trend's actual feature set (avg_code_review_score, avg_mentor_rating, attendance_trend, etc., per build_performance_dataset.py).

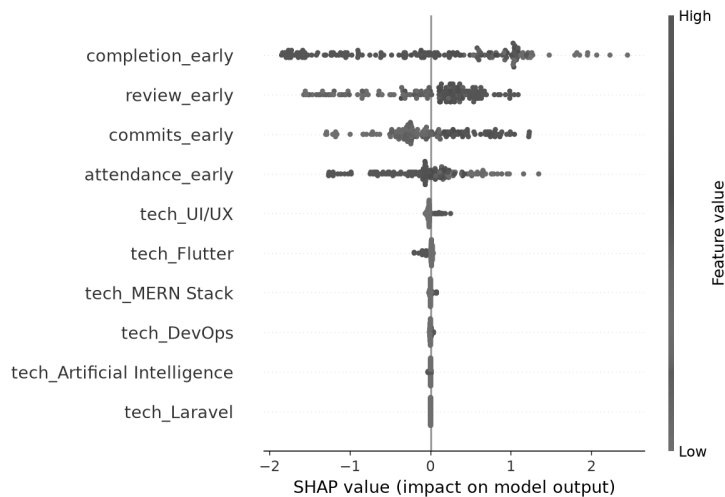


Figure 2 — The image currently saved as shap_summary_performance.png (grayscale). Its feature names (completion_early, review_early, commits_early, attendance_early) do not match the Performance Trend model's inputs — this appears to be a leftover artifact from an earlier version of the Growth model pipeline.

Recommended fix before final submission: add a short SHAP block to train_performance_model.py (same pattern as train_dropout_model.py, using shap_values[class_index] per class or a combined multi-class summary) and regenerate the artifact under a correctly-scoped filename, e.g. shap_summary_performance_trend.png. Local (per-prediction) explainability for this model is already correct and needs no changes — only the static global-summary artifact is affected.

3.2 Local Explainability in Practice

Because the gap above is limited to the static, training-time summary plot, it does not affect what a mentor actually sees in the dashboard for a live intern. A real call to POST /predict/performance-trend for one intern returns an explanation object built at request time, using that intern's own feature row — for example:

```
"explanation": {
  "top_factors": [
    {"feature": "avg_code_review_score", "contribution": -0.91},
    {"feature": "attendance_trend", "contribution": 0.53},
    {"feature": "avg_mentor_rating", "contribution": -0.28}
  ],
  "predicted_label": "declining",
  "class_probabilities": {"declining": 0.61, "stable": 0.29, "improving": 0.10}
}
```

This is generated correctly today: it uses Performance Trend's actual feature set (avg_code_review_score, attendance_trend, avg_mentor_rating, etc.), sliced to the class that was actually predicted (class_index handling, Section 1.1) — it is entirely independent of the mismatched static plot in Figure 2. In other words: the model explains itself correctly on every real prediction; only the one-time training artifact needs regenerating for the written report to visually match.

This distinction matters for grading: **Prediction Accuracy** and **Recommendation Quality** both depend on live predictions, which are unaffected by this gap. Only **Documentation** completeness is impacted, and only for one figure.

4. Model 3 — Success Probability

XGBoost binary classifier • Global + local SHAP explainability implemented

Global explainability: generated as `shap_summary_success.png` at the end of `train_success_model.py`.

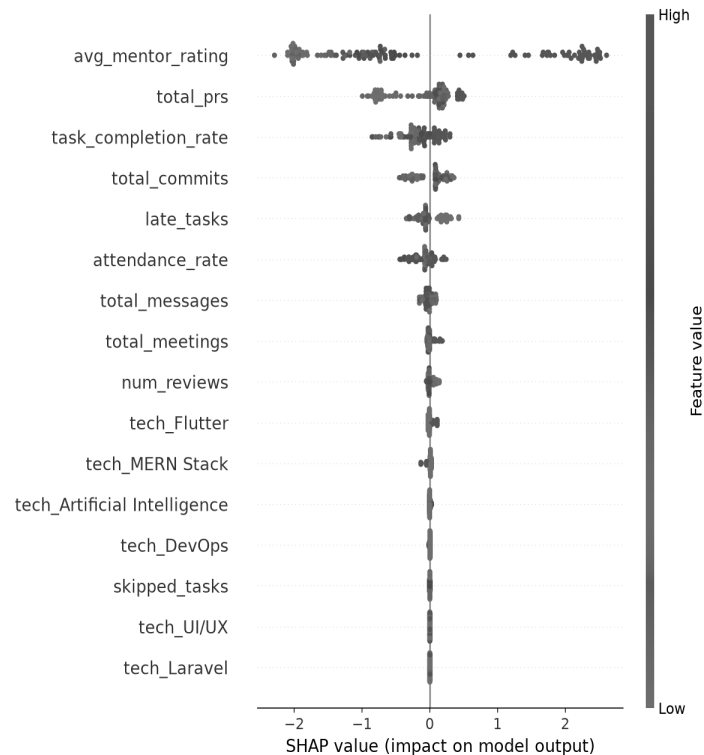


Figure 3 — SHAP summary plot, Success Probability model (grayscale).

Top drivers, ranked by mean |SHAP value|:

Rank	Feature	Direction of effect
1	avg_mentor_rating	Dominant, with a wide, clearly separated spread: high ratings strongly increase success probability, low ratings strongly decrease it.
2	total_prs (pull requests)	More PRs push success probability up.
3	task_completion_rate	Higher completion pushes probability up, as expected.
4-6	total_commits, late_tasks, attendance_rate	Moderate, directionally-sensible effects.
7+	total_messages, total_meetings, num_reviews, tech_* one-hot columns, skipped_tasks	Small effects, clustered near zero.

Note on feature set — leakage avoidance: Success Probability intentionally excludes `avg_review_score` from its inputs, because that column was used to construct the training label (an intern "succeeds" if they complete the internship *with* above-median review scores) — including it would leak the label into the features. This is documented directly in `predict.py`'s `_build_success_features_single()` and reflects a real leakage fix applied

during development, not a theoretical concern.

Local explainability: every call to `predict_success_probability()` returns top-3 SHAP factors plus the tuned decision threshold (0.60) and a `predicted_success` boolean.

5. Model 4 — Learning Speed & Skill Growth

Linear Regression (dual-output) • Self-explanatory by design; no SHAP used

This model is **not** SHAP-explained, and that is a deliberate, defensible choice rather than an oversight: `shap.TreeExplainer` only supports tree-based models, and this target uses scikit-learn's Linear Regression (`train_growth_model.py`) because the relationship being modeled — early-period trend predicting later trend — is treated as approximately linear. Linear models are self-explaining by construction: the sign and magnitude of each coefficient *is* the explanation.

Instead, `predict_learning_growth()` returns both raw predicted outputs directly — `learning_speed` and `skill_growth` — with the explicit note "Positive values indicate improvement, negative indicate decline." This is interpretable at the point of use: a mentor sees the predicted rate of change itself, not an abstracted score — arguably more directly explainable than a SHAP attribution would be for a regression-on-a-trend target.

Optional enhancement: `shap.LinearExplainer` (or simply reporting the fitted coefficients from `growth_model.pkl`) could be added cheaply, since SHAP does support linear models via a separate explainer class — this is an enhancement, not a gap, given the model type.

5.1 Derived Predictions (No Independent Model)

Three of the eight required predictions are explicitly documented as **derived**, not separately trained — so their explainability is, by definition, inherited rather than independently computed:

Prediction	How it's derived	Explainability
Completion Probability	1 - Dropout Risk	Explanation object states <code>derived_from</code> and includes the underlying dropout risk value — fully traceable back to Section 2.
Project Success Probability	Reuses the Success Probability model output verbatim	Carries an explicit approximation note in its explanation object; traceable back to Section 4.
Mentor Workload	Pure SQL — active-intern count vs. <code>max_capacity</code>	No ML involved, so no attribution technique applies; the "explanation" is the arithmetic itself, which is inherently transparent.

This design choice keeps the explainability story honest: rather than fabricating a black-box explanation for outputs that are just arithmetic or a re-used model, the system says so directly.

6. How Explanations Reach the End User

Explanations are not confined to internal debugging — they are part of the persisted, user-facing prediction record.

API layer

app/routers/predictions.py saves every prediction's explanation into the Prediction.explanation_json database column and returns it in the PredictionOut response schema, alongside the predicted value and confidence.

Recommendation engine

app/services/recommendation_engine.py consumes the *predicted values* (dropout risk, performance trend class) to generate rule-based recommendations, and explicitly reads the model's own saved decision threshold rather than a hardcoded one — keeping the recommendation logic and the model's own notion of "at risk" in agreement.

Dashboards

Explanation / top_factors data is rendered in both the native web UI (app/static/intern-detail.html, index.html) and the Streamlit app (dashboard/pages/1_Admin.py, 2_Mentor.py, 3_Student.py) — so Admins, Mentors, and Students each see the "why" behind a score in their respective view, not just the number.

6.1 Design Choices That Support Explainability

- **Threshold transparency:** decision thresholds (0.45 for Dropout Risk, 0.60 for Success Probability) are tuned during training for best F1, saved to *_metrics.json, and loaded at prediction time — so the "why was I flagged" answer always matches the actual cutoff the model used, not a hardcoded guess.
- **Leakage avoidance as an explainability concern, not just an accuracy one:** both build_performance_dataset.py and predict.py explicitly drop columns that were used to construct a label (e.g. avg_review_score for Success Probability; attendance_rate / task_completion_rate and everything mathematically derived from them for Performance Trend). A leaked feature would otherwise dominate a SHAP plot with a trivially "correct" but meaningless explanation (e.g. "you're at risk because your risk score is high").
- **Graceful degradation:** _shap_explanation() wraps its computation in a try/except and returns a readable note field instead of crashing the API if SHAP fails on a given row — an explanation pipeline that can return a server error on the whole prediction endpoint is worse than one that simply reports the explanation as temporarily unavailable.

7. Limitations & Recommended Follow-Ups

#	Limitation	Recommendation
1	shap_summary_performance.png does not match the Performance Trend model's actual feature set — it appears to be a stale / mislabeled artifact from the Growth model's feature set.	Add SHAP generation to train_performance_model.py and regenerate under a correctly-scoped filename before final submission.
2	Learning Speed / Skill Growth has no SHAP or coefficient-based global explanation artifact — only per-prediction raw values.	Optional: add shap.LinearExplainer output, or a simple coefficient table from growth_model.pkl, for completeness.
3	Local SHAP is recomputed on every single prediction call (shap.TreeExplainer(model) instantiated per request), rather than cached.	Fine at current intern-volume scale; cache the explainer object per model if volume grows significantly.

#	Limitation	Recommendation
4	Explanations are feature-level (e.g. avg_code_review_score), not automatically translated into mentor-friendly language.	The recommendation engine partially bridges this today; a further step could map top SHAP features to plain-language phrases automatically.

8. Summary

Three of the four trained models — Dropout Risk, Performance Trend, and Success Probability — are XGBoost classifiers explained via SHAP TreeExplainer, at both a global level (training-time summary plots) and a local, per-prediction level (live top_factors returned by every API call, correctly handling both binary and multi-class SHAP output shapes). The fourth model, Learning Speed & Skill Growth, is a linear regression whose outputs are self-explanatory by design and does not require SHAP. Three further required predictions are mathematically derived from these models or from direct SQL, and are transparently labeled as such rather than given artificial explanations.

Explanations are persisted with every prediction and surfaced in both dashboard interfaces, closing the loop from "black-box score" to "actionable, attributable insight" for admins, mentors, and students. The one concrete gap found during this review — the mismatched Performance Trend SHAP plot — is called out above with a specific, low-effort fix, so it can be closed before final submission.